UNIVERSIDAD PONTIFICIA ICAI ICADE COMILLAS MADRID

Sistemas de Almacenamiento NoSQL. HBase. Data Modeling

Máster en Big Data. Tecnología y Analítica Avanzada (MBD)

Escuela Técnica Superior de Ingeniería (ICAI)

comillas.edu

Índice

- Diseño de tablas en HBase
  - Consideraciones generales de diseño
  - Diseño centrado en aplicación
  - Diseñando rowkeys
  - Otras características de las tablas en HBase

comillas.edu

Consideraciones

- La información de una familia se almacena en un único directorio en HDFS
- Todas las columnas de una familia se almacenan juntas en disco y **ordenadas** lexicográficamente.
- Columnas con diferentes patrones de acceso se deben almacenar en diferentes familias.
- Columnas con diferentes características de almacenamiento deben ir a familias distintas.
- Intentar obtener la información en una única llamada al API.

comillas.edu

Vista almacenamiento físico

Row key	Column qualifier	Cell value	Timestamp **(long)**
Row1	CF1:colA	value3	*time7*
Row1	CF1:colA	value2	*time5*
Row1	CF1:colA	value1	*time1*
Row10	CF1:colA	value1	*time4*
Row 10	CF1:colB	value1	*time4*

- Las familias se almacenan separadamente
- Claves y cualificadores se almacenan ordenados lexicográficamente

comillas.edu

Diseño de tablas

Column Families:

- Agrupar columnas que se consultan juntas. Cada familia se define al crear la tabla y afecta la forma en que se almacenan los datos en HFiles.

Modelo Schemaless:

- HBase permite agregar columnas de forma dinámica (qualifiers) dentro de cada familia, lo que brinda flexibilidad, pero requiere un diseño cuidadoso para evitar inconsistencias.

Optimización y Normalización:

- Separar la información en diferentes tablas si es necesario, según patrones de acceso (por ejemplo, tablas separadas para transacciones y para auditoría).

comillas.edu

Diseño de rowkeys

- La RowKey es la base para la distribución de datos en el clúster, ya que determina en qué regiones se almacenan los datos.
- Una buena RowKey evita la formación de “hot spots” y permite consultas eficientes.

Estrategias de Diseño de RowKeys:

- Hashing
- Salting (Agregación de Prefijo Aleatorio)
- Timestamp Inverso
- Clave compuesta

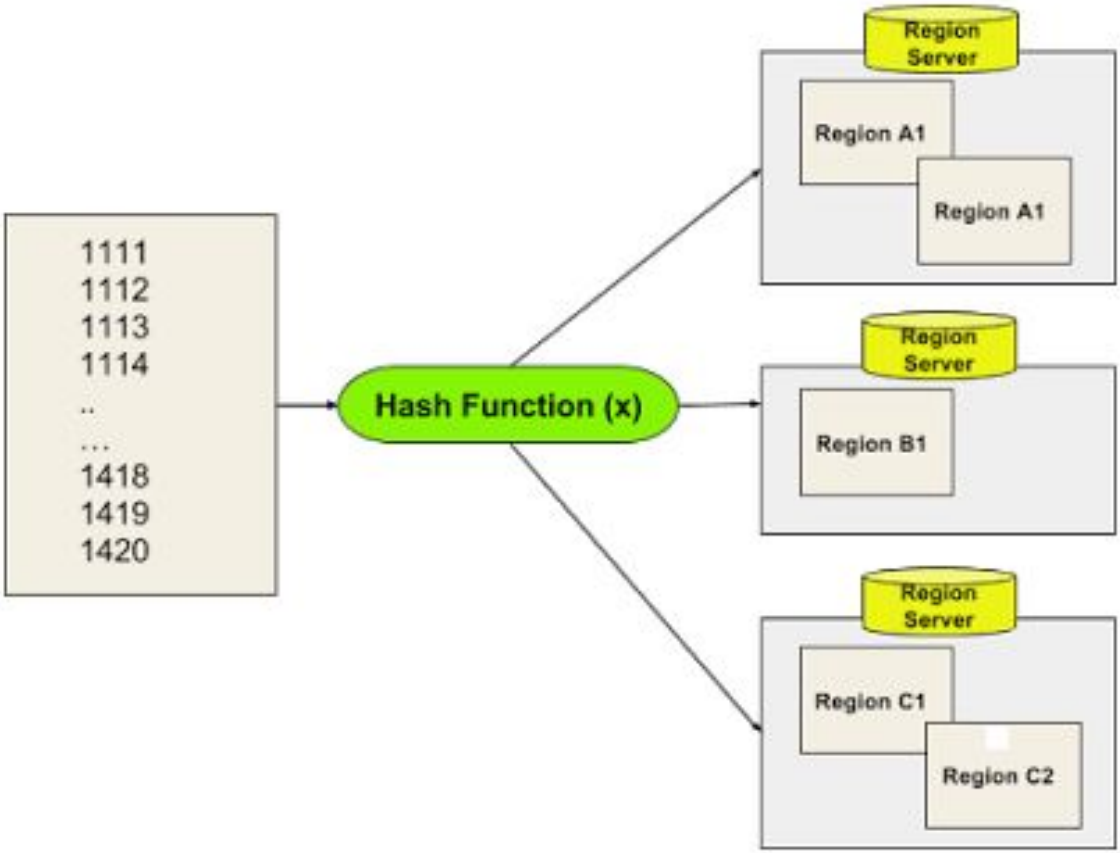
comillas.edu

Hotspotting

- Ocurre cuando una gran cantidad de tráfico de los clientes se dirige a un único regionserver, o unos pocos. El tráfico puede ser de lecturas o escrituras.
- Relacionado con la gestión de claves y distribución en regiones. Recordad que las claves en HBase se almacenan ordenadas lexicográficamente.
- Generado por claves monótonamente crecientes.
- Para prevenirlo:
  - Claves aleatorias
  - Claves compuestas
  - Claves con prefijo
  - Inversión de la clave

comillas.edu

Claves Aleatorias. Hashing



comillas.edu

Claves Aleatorias. Hashing

- Hay que aplicar una función de hash en la clave para la lectura y escritura. Mayor coste computacional.
- La clave hash puede no ser reversible. La información no es recuperable.
- Las claves hash suelen ser de longitud fija y largas. Mayor consumo de espacio en disco.
- No tiene sentido las búsquedas parciales ni por intervalo de clave. Las operaciones de paginado no se utilizan.
- Devuelve resultados sin ordenar. Si es necesario hay que ordenar los resultados.

comillas.edu

Claves compuestas

Clave	Descripción
<UsusarioId>	Se busca por el id del usuario
<UsusarioId>-<Date>	Se busca por el id del usuario y fecha
<UsusarioId>-<Date>-<PedidoId>	Se busca por usuario y fecha y pedido
<UsusarioId>-<Date>-<PedidoId>-<Articulo>	Se busca por usuario y fecha y pedido y artículo

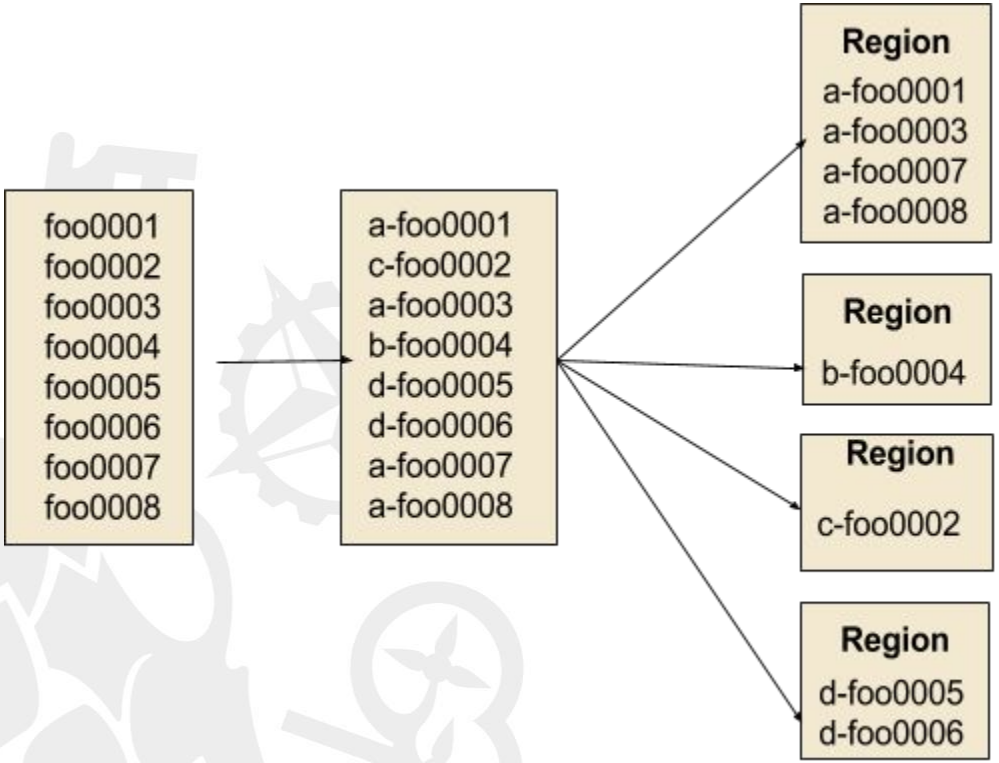
comillas.edu

Claves compuestas

- Por si mismo no resuelve el problema de hotspotting. Hay que aplicar otras técnicas a cada componente.
- Mayor consumo de espacio en disco.
- Si tiene sentido las búsquedas parciales por intervalo de clave. Las operaciones de paginado suelen ser necesarias.
- Devuelve resultados ordenados según el diseño de la clave. Si es necesario otra ordenación hay que gestionarla por separado.

comillas.edu

Claves con prefijo - Salting



comillas.edu

Salting

Añade un prefijo aleatorio (o basado en hash) al inicio de la RowKey para distribuir la carga.

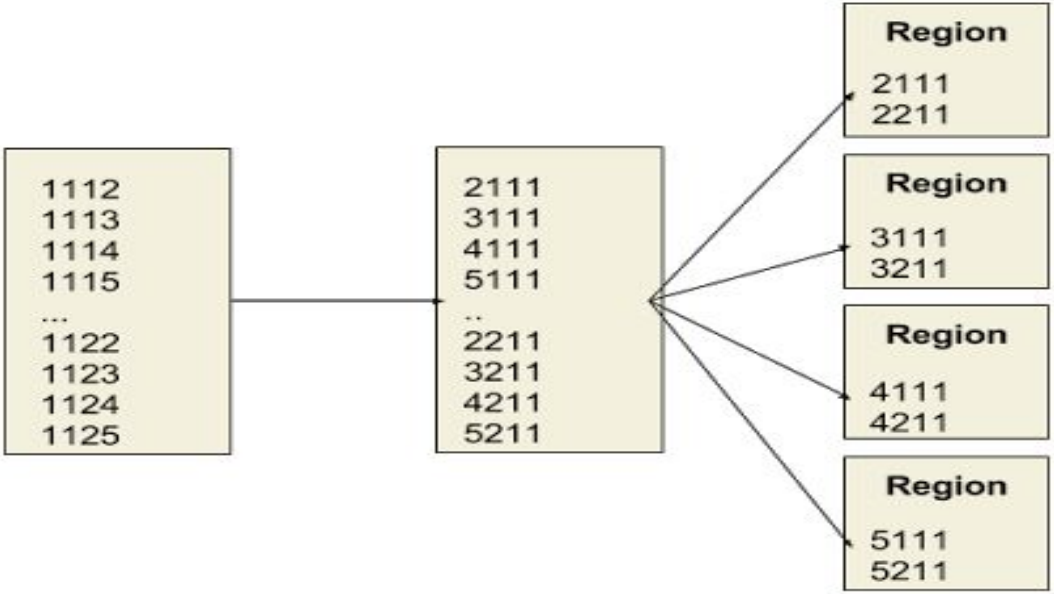
Genera problemas cuando se modifica el número de RegionServers

Las búsquedas son más costosas. El procesamiento adicional puede estar en el lado del cliente o en el de hbase.

Devuelve resultados ordenados por cada región.

comillas.edu

Inversión de la clave



comillas.edu

Inversión de la clave

- Puede no generar distribuciones homogéneas.
- No tiene sentido las búsquedas parciales ni por intervalo de clave. Las operaciones de paginado no se utilizan.
- Devuelve resultados sin ordenar por la clave natural.
- Utiliza la resta del timestamp actual a un valor máximo (por ejemplo, Long.MAX_VALUE) para que los registros más recientes aparezcan primero.

comillas.edu

Contadores

Los contadores en HBase permiten realizar incrementos atómicos sobre valores numéricos.

Se utilizan para llevar el seguimiento de estadísticas, visitas, clics, entre otros, sin necesidad de mecanismos transaccionales complejos.

HBase provee el comando increment para aumentar el valor de una celda en una cantidad determinada de forma atómica.

Internamente, la operación de incremento garantiza la consistencia utilizando el mecanismo de escritura en WAL y la coordinación entre RegionServers.

```
increment 'practica:estadisticas', 'cliente123',
'contador:visitas', 1
```

No requiere una transacción completa, lo que reduce la sobrecarga.

Es útil en entornos de alta concurrencia donde múltiples clientes pueden intentar actualizar el mismo contador simultáneamente.

comillas.edu

MOBS

Los MOBs (Medium Objects) son una característica de HBase para gestionar objetos de tamaño medio que son demasiado grandes para almacenarse de manera eficiente en una celda normal, pero que no son tan grandes como para ser tratados como archivos externos.

Permiten almacenar valores de celda que superan un cierto umbral (configurable) en un área de almacenamiento separada, optimizando la lectura y escritura.

- Cuando el tamaño de un valor supera el umbral definido (por ejemplo, 100 KB o un valor personalizado), HBase puede almacenar el contenido en un archivo MOB.
- La celda en la tabla contiene un puntero al archivo MOB, y HBase se encarga de gestionar la lectura/escritura de estos objetos de manera eficiente.
- La política de retención y la limpieza de MOBs se gestionan a través de parámetros de configuración y se integran en el proceso de compactación.

comillas.edu

Elevado número de familias

- Recomendable mantener sobre 3.
- Familias agrupan información del mismo tipo y/o el mismo patrón de acceso
  - Compresión
  - Diferente cardinalidad de familias
- Memstore se comparte entre todas las CF de una Region.
- Compactación. Cada familia se almacena en ficheros independientes. Compactación menor se dispara con tres ficheros.
- Split. Cardinalidad diferente genera archivos muy pequeños.
- Muchas entradas en hbase:meta

comillas.edu